

Obserwator, inne nazwy: **Obiekty zależne** (ang. dependents), **publikuj-subskrybuj** (publish-subscribe)
Jest wiele koncepcji tego wzorca. Poniżej podano najbardziej istotne minimum.

Co to jest?

- Podmiot informuje Obserwatorów, którzy zarejestrowali się wcześniej u Podmiotu na listę powiadomień.
- Znany z social mediów model zależności publisher-subscribers, gdzie obiekty śledzące inny obiekt (subskrybujące powiadomienia), dostaną automatycznie informację (tylko!) o zmianie stanu obiektu śledzonego. Przykład: Pojawił się nowy filmik na ulubionym kanale? - Wszyscy subskrybenci dostaną powiadomienie, ale kto chce go pobrać, musi zrobić to sam.
- Model luźnej zależności między obiektami, gdzie zmiana jednego z nich, inicjuje jednokierunkowo potencjalne zmiany/ akcje w innych obiektach. Obiekt Podmiot nie wymusza zmian u Obserwatorów, tylko informuje. Obserwatorzy sami są odpowiedzialni za reakcję/ obsługę - czy i w jaki sposób zareagują, to zależy od ich własnej implementacji/ logiki biznesowej.
- Obiekty Obserwator nie muszą być identyczne i mogą reagować w unikalny dla siebie sposób na to samo powiadomienie.

Użyjesz Observera, gdy:

- Podmiot
 - Dokonuje zmian swojego stanu w sposób nieregularny, nieprzewidywalny dla Obserwatorów, ale zwykle natychmiast po pojawieniu się zmian
 - Nie czeka na potwierdzenie otrzymania powiadomienia od Obserwatorów, ani na potwierdzenie, który z nich pobrał zmieniony stan podmiotu - wyślij i zapomnij
 - Panuje nad tym, co udostępnia Obserwatorom - może udostępniać cały swój stan lub fragment
- Podmiot wobec Obserwatorów
 - Nie zarządza Obserwatorami, nie może ich usuwać, dodawać, blokować, priorytetować, sortować, nadawać przywilejów
 - Wszyscy Obserwatorzy są anonimowi i równoprawni - nie ma VIP-ów, kart stałego klienta itp.. - powiadamianie jest w kolejności losowej, niekoniecznie tej samej za każdym razem
 - Nie ma ograniczeń na liczbę Obserwatorów, ani na ich typ - każdy może się zarejestrować,
 - Ilość Obserwatorów zmienia się w czasie, Podmiot nie ma na to wpływu
 - Nie zna sposobów reakcji na powiadomienie ani na wykorzystanie pobranych danych z Podmiotu
 - Nie rejestruje (więc nie ma wiedzy), kto z Obserwatorów zareagował na powiadomienie i pobrał jego stan, a kto zignorował - ew. może zliczać ilość pobrań
- Obserwatorzy wobec Podmiotu
 - Nie śledzą aktywnie Podmiotu, nie czekają aktywnie na powiadomienie
 - W zależności od swojej logiki biznesowej mogą:
 - w każdej chwili zrezygnować z otrzymywania powiadomień - wyrejestrować się
 - w każdej chwili zarejestrować się ponownie
 - nie muszą zareagować na powiadomienie,
 - nie muszą pobrać aktualny stan Podmiotu
 - Gdy pobiorą stan podmiotu:
 - przechowują go u siebie lokalnie,
 - mogą, ale nie muszą podjąć jakąś akcję lub podjąć ją warunkowo w zależności od zawartości otrzymanych danych
 - mogą otrzymywać powiadomienia od wielu Podmiotów - wzorzec rozbudowany
 - nie mają pełnej wiedzy o Podmiocie, a tylko tę, jaką mogą pobrać w odpowiedzi na powiadomienie

Korzyści:

- Racjonalna, niezobowiązująca komunikacja
 - ograniczenie ruchu w sieci, bo przesyłane są tylko krótkie info o zmianie, a nie cała zmiana
 - ograniczenie aktywności Obserwatorów, którzy nie muszą synchronicznie sprawdzać (np.co godzinę), co nowego u Podmiotu - realizują swoje zadania, aż otrzymają powiadomienie od Podmiotu
 - Podmiot nie spamuje wszystkich możliwych Obserwatorów o zmianie swojego stanu, ale informuje tylko zarejestrowanych
 - Obserwator, nawet ten zarejestrowany, nie musi zareagować na powiadomienie, ani pobrać danych
- Izolacja implementacji Podmiotu i Obserwatorów
 - Działanie Podmiotu nie zależy od Obserwatorów i wzajemnie
 - Podmiot nie zna implementacji Obserwatora, i wzajemnie - komunikują się interfejsami
- Obserwator decyduje:
 - może w runtime, rejestrować się lub wyrejestrować, w zależności od innych czynników i realizowanej logiki biznesowej

- Podmiot nie zarządza Obserwatorami (nie ma takiej możliwości), nie śledzi ich aktywności:
 - Rejestracji i deregistracji
 - Pobrania stanu Podmiotu

Wady:

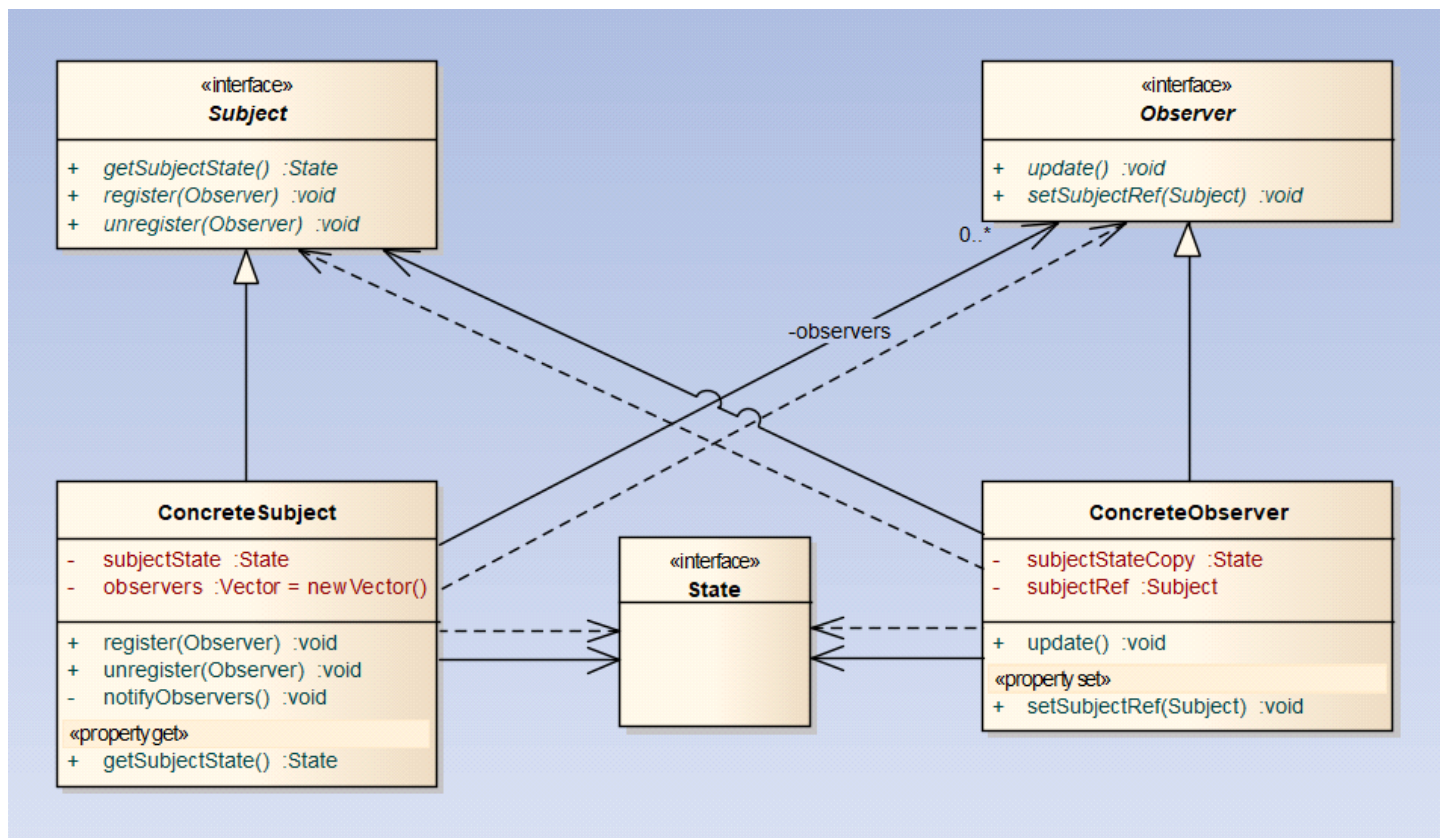
- Jednokierunkowy strumień powiadomienia: od Podmiotu do Obserwatora, bez potwierdzenia odbioru - Podmiot wie, kogo powiadomił, ale nie wie, kto z powiadomionych pobrał jego aktualne dane.
- Podmiot nie wie, jak została zaimplementowana metoda update() w Obserwatorze - nie zna wszystkich skutków swojej aktualizacji.
- Anonimowość równych Obserwatorów - W standardowej wersji wszyscy oni mają równy status, nie ma VIP-ów i priorytetów, jest losowa kolejność powiadamiania, brak możliwości ustalenia tożsamości Obserwatorów przez Podmiot
- Podmiot nie ma wpływu na to, kto go obserwuje, brak możliwości usuwania niechcianych Obserwatorów, brak możliwości dodania przez Podmiot pożądanych Obserwatorów - Obserwatorzy są rejestrowani i wyrejestrowywani poza kontrolą Podmiotu - Podmiot tym nie zarządza.
- Podmiot nie ma żadnego wpływu na to, jak zostaną wykorzystane jego dane.

Porównanie do innych wzorców:

- Mediator - wprowadza obiekt pośredniczący, który może zarządzać komunikacją, filtrować Obserwatorów - w Obserwatorze komunikacja jest jednostronna, bezpośrednia i anonimowa.
- Most -
 - W Moście - główny wykonawca deleguje podrzędne funkcjonalności do wykonania podwykonawcom i oczekuje od nich potwierdzenia lub dostarczenia wyniku;
 - w Obserwatorze Podmiot niczego nie oczekuje od Odbiorców - informuje ich, a co oni zrobią, to ich sprawa. Zmiana stanu Podmiotu nie zależy od Obserwatorów. Ich obecność lub nieobecność nie ma żadnego wpływu na jego logikę biznesową, co najwyżej (przy bardzo długiej liście Obserwatorów) na obniżenie wydajności.

Jak to zrobić?

- Wzorzec zawiera 4 podstawowe obiekty + piąty interfejs Stanu Podmiotu:
 - **Subject/** Podmiot - interfejs do podmiotu udostępniany Obserwatorom, którzy dzięki niemu mogą wpisywać/wypisywać się na listę wysyłkową
 - **Observer/** Obserwator - interfejs do powiadamianych obiektów
 - **ConcreteSubject/** KonkretnyPodmiot - klasa podmiotu, który:
 - Implementuje interfejs Podmiot
 - Przechowuje wyróżniony stan podmiotu (istotny dla obserwatorów), który może zostać zmodyfikowany w runtime
 - Przechowuje listę obserwatorów,
 - Powiadamia Obserwatorów po każdej modyfikacji wyróżnionego stanu
 - **ConcreteObserver/** KonkretnyObserwator:
 - Implementuje interfejs Obserwator
 - Zawiera referencję do obiektu ConcreteSubject (widzianego przez maskę interfejsu Subject)
 - Przechowuje stan, powiązany ze stanem ConcreteSubject, aktualizowany na podstawie stanu ConcreteSubject
 - **State** - interfejs opisuje stan podmiotu, udostępniany Obserwatorom, nie musi zawierać pełnej informacji o obiekcie stanu Podmiotu, ale tylko te, które mają być udostępniane Obserwatorowi
- W zależności od koncepcji wzorca, zamiast interfejsu Obserwator może być klasa abstrakcyjna z polem "stanPodmiotu"
 - jednak jest to rozwiązanie mniej uniwersalne, lepiej niech każdy konkretny Obserwator sam decyduje, jak przechowuje stan Podmiotu.



Przykład autorskiej implementacji

<https://github.com/wblonski/pattern-observer.git>